



by Ashok Bhattarai

Servlets and Java Server Pages

Unit 7: Web Technologies for Enterprise Applications

Learning Objectives



Explain servlets and server pages



Demonstrate JDBC and ODBC connectivity



Demonstrate servlet programming



Understand JSP fundamentals

Web Container & Servlets



Web container

- introduction to servlets
- lifecycle of servlets



Servlets

- APIs
- writing servlet programs
- readinform parameters
- processing forms
- handling HTTP request and response (GET / POST request)
- Database access with servlets
- handling cookies

Servlet vs JSP & JSP Access Model

Servlet vs JSP

- JSP access model
- JSP syntax (directives, declarations, expression, scriptlets, comments)

JSP Specifics

- JSP implicit objects
- object scope
- processing forms
- database access with JSP

Practical Work



Creating forms



Processing forms



JDBC connectivity



ODBC connectivity

Key Takeaways

1 Servlets as Core Server-Side Components

Servlets provide powerful Java-based server-side programming for handling HTTP requests, managing sessions, and interacting with web containers.

3 Essential Database Connectivity

Both Servlets and JSPs leverage JDBC and ODBC for robust database interactions, enabling data-driven enterprise web applications.

2 JSP for Dynamic Content Presentation

JSP simplifies the creation of dynamic web content by embedding Java code directly into HTML, streamlining the view layer of web applications.

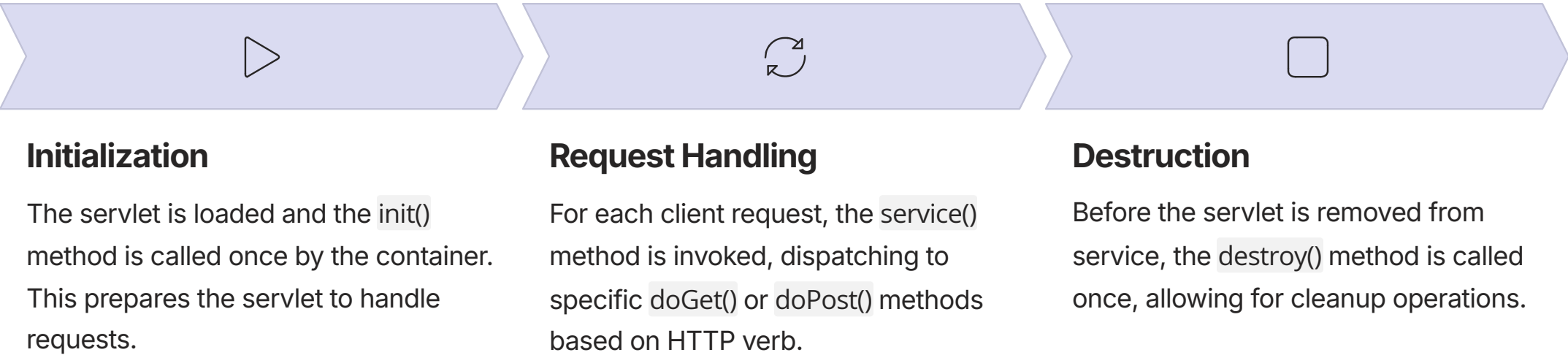
4 Foundations for Enterprise Web Development

Mastering Servlets and JSP is crucial for building scalable, secure, and maintainable web applications for complex business environments.

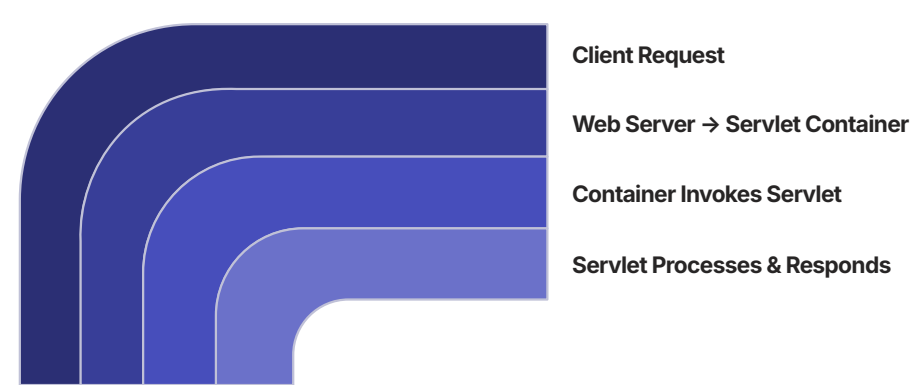
Servlet Lifecycle & Architecture

Understanding the Servlet lifecycle and its architecture is fundamental for developing robust and efficient web applications. It defines how servlets are initialized, handle client requests, and are eventually destroyed by the web container, providing a structured approach to server-side programming.

Servlet Lifecycle & Key Methods



Request/Response Flow & Container Responsibilities



The diagram illustrates how a client's request travels through the web server and servlet container to reach the servlet, and how the response is routed back, highlighting the container's crucial role in managing this interaction.



Container Responsibilities

- Manages servlet lifecycle (loading, initialization, invocation, destruction).
- Maps URL to appropriate servlet.
- Handles HTTP request parsing and response formatting.
- Provides runtime environment and security services.
- Supports session management and concurrency.

The web container (e.g., Apache Tomcat) plays a central role, abstracting away low-level network programming and allowing developers to focus on business logic.

HTTP Protocol & Request/Response

The Hypertext Transfer Protocol (HTTP) is the foundation of data communication for the World Wide Web. It's a stateless protocol that enables clients (like web browsers) to request resources from servers. Understanding its core components – methods, request/response structures, and status codes – is crucial for any web developer.

HTTP Methods

These verbs define the type of action a client wishes to perform on a resource:

Method	Description
GET	Retrieves data from a specified resource without altering server state.
POST	Submits data to be processed to a specified resource, often creating a new resource.
PUT	Updates a resource or creates one if it doesn't exist at the specified URI.
DELETE	Removes a specified resource from the server.
HEAD	Requests headers identical to a GET request, but without the response body.

HTTP Request Structure



Request Line

Method (e.g., GET), URL, and HTTP version.



Headers

Key-value pairs providing metadata (e.g., Content-Type, Authorization, User-Agent).



Body

Optional data payload for POST or PUT requests (e.g., JSON, form data).

HTTP Response Status Codes

Servers respond with a status code indicating the outcome of the request:

Category	Meaning	Example
1xx	Informational	100 Continue
2xx	Success	200 OK
3xx	Redirection	301 Moved Permanently
4xx	Client Error	404 Not Found
5xx	Server Error	500 Internal Server Error

Request vs. Response

Request	Response
Initiated by the client (browser, app).	Sent by the server to the client.
- Asks for a resource or action. - Contains method, URL, headers, optional body. - Example: User clicks a link.	- Provides the requested resource or status. - Contains status line, headers, optional body (e.g., HTML, JSON). - Example: Server sends back a webpage.

Servlet API & Core Classes

The Servlet API provides the fundamental building blocks for developing dynamic web applications in Java. Understanding its core interfaces and classes is essential for managing client requests, generating responses, and configuring servlet behavior within a web container.

Core Interfaces: `ServletRequest` & `ServletResponse`

These interfaces define the general contract for all request and response objects, regardless of the underlying protocol. They provide basic methods for reading client data and writing server responses.

<code>ServletRequest</code> Represents a client request to the servlet. It provides methods to read incoming parameters, headers, and input streams. - Access input data (parameters, streams) - Retrieve client information (IP address, host) - Manage request attributes	<code>ServletResponse</code> Represents the server's response back to the client. It provides methods for sending output data, setting headers, and managing status codes. - Send output data (text, binary) - Set response headers (Content-Type) - Manage response buffering and commit
---	---

HTTP-Specific Implementations

For web applications utilizing the HTTP protocol, the Servlet API provides specialized classes that extend the generic interfaces, adding HTTP-specific functionalities.

<code>HttpServletRequest</code> Extends <code>ServletRequest</code> with methods tailored for HTTP requests, such as: <code>getMethod()</code>: Returns the HTTP method (GET, POST, etc.) <code>getRequestURI()</code>: Gets the URI of the request <code>getSession()</code>: Accesses the HTTP session <code>getParameter(String name)</code>: Retrieves a single parameter value <code>getParameterValues(String name)</code>: Retrieves multiple values for a parameter (e.g., from checkboxes) <code>getHeader(String name)</code>: Gets the value of a specific HTTP header <code>setAttribute(String name, Object value)</code>: Stores an object in the request scope <code>getAttribute(String name)</code>: Retrieves an object from the request scope	<code>HttpServletResponse</code> Extends <code>ServletResponse</code> with methods for handling HTTP responses, such as: <code>setStatus(int sc)</code>: Sets the HTTP status code <code>sendRedirect(String location)</code>: Redirects the client to another URL <code>addCookie(Cookie cookie)</code>: Adds a cookie to the response <code>setContentType(String type)</code>: Sets the Content-Type header <code>getWriter()</code>: Returns a <code>PrintWriter</code> for sending character text
--	--

```
String username = request.getParameter("username");
String userAgent = request.getHeader("User-Agent");
request.setAttribute("message", "Welcome!");
```

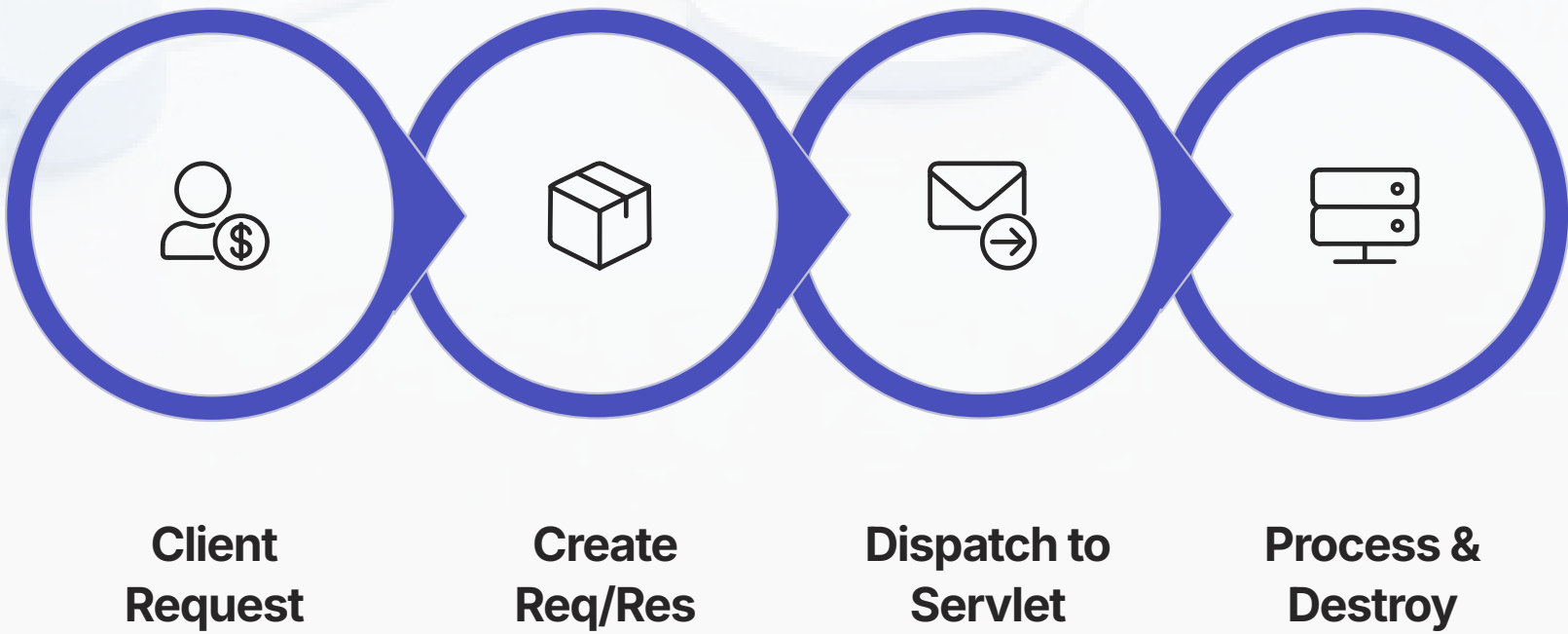
Servlet Configuration & Context

Servlets interact with their environment through configuration and context objects, providing access to initialization parameters and web application resources.

 <code>ServletConfig</code> An object representing the initialization parameters for a specific servlet. It is unique to each servlet instance. - Obtain servlet-specific <code>init-param</code> defined in <code>web.xml</code> - Access the <code>ServletContext</code> object	 <code>ServletContext</code> An object representing the entire web application. It is shared among all servlets within the same web application. - Access application-wide <code>context-param</code> from <code>web.xml</code> - Share attributes among servlets (<code>setAttribute</code>, <code>getAttribute</code>) - Log messages to the container's log files - Obtain resources (files, JSPs) within the web application
---	--

Request & Response Object Lifecycle

Understanding the lifecycle of these objects is key to proper resource management and statelessness in web applications.



Form Processing & Data Handling

Web forms are fundamental for user interaction, enabling data collection and submission. Servlets provide robust mechanisms to process this data, requiring careful attention to validation and security.

HTML Form Methods: GET vs. POST

The choice of HTTP method dictates how form data is sent to the server, impacting visibility, security, and data capacity.

GET Method

- Appends form data to the URL as query parameters.
- Visible in the browser's address bar and history.
- Bookmarkable and shareable.
- Limited data size (browser/server dependent, typically ~2048 characters).
- Suitable for data retrieval (e.g., search queries) where data is not sensitive.

```
<form action="/search" method="GET">
  <input type="text" name="query">
  <button type="submit">Search</button>
</form>
```

POST Method

- Sends form data within the body of the HTTP request.
- Not visible in the URL, browser history, or server logs.
- Not bookmarkable or directly shareable.
- No practical data size limit.
- Essential for submitting sensitive data (passwords, financial info) or large files.
- Used for operations that change server state (e.g., creating, updating resources).

```
<form action="/submit-order" method="POST">
  <input type="text" name="item_name">
  <input type="number" name="quantity">
  <button type="submit">Place Order</button>
</form>
```

Reading Form Data in Servlets

`HttpServletRequest` provides methods to access incoming form parameters, both single and multiple values.

Single Value Parameters

Use `request.getParameter(String name)` to retrieve the value of a form field. If multiple values exist for the same name, it returns only the first one.

```
// For a text input: <input type="text" name="username">
String username = request.getParameter("username");

// For a radio button: <input type="radio" name="gender" value="male">
String gender = request.getParameter("gender");
```

Multiple Value Parameters

For fields that can have multiple selected values (e.g., checkboxes, multi-select dropdowns), use `request.getParameterValues(String name)`, which returns an array of strings.

```
// For checkboxes: <input type="checkbox" name="hobbies" value="reading">
String[] hobbies = request.getParameterValues("hobbies");

if (hobbies != null) {
    for (String hobby : hobbies) {
        System.out.println("Selected hobby: " + hobby);
    }
}
```

Validation & Security Best Practices

Form Validation Techniques

Ensuring data integrity and correctness is crucial.

- **Client-side Validation:** HTML5 attributes (`required`, `pattern`), JavaScript. Provides immediate feedback to the user but is easily bypassed.
- **Server-side Validation:** **Absolutely essential.** Always re-validate data on the server. Checks for data type, length, format (regex), range, and business rules.
- **Feedback:** Provide clear, user-friendly error messages for invalid input.

Security Considerations

Protecting against common web vulnerabilities.

- **Input Validation:** Sanitize and validate all user input to prevent injection attacks (SQL, Command, LDAP).
- **XSS Prevention:** Encode all output displayed to the user (e.g., HTML escape). Never render raw user-provided data directly.
- **CSRF Protection:** Implement CSRF tokens for state-changing requests to prevent unauthorized actions by authenticated users.
- **Sensitive Data Handling:** Use POST for sensitive data, encrypt data in transit (HTTPS), and hash passwords.



Session Management & Cookies

HTTP Stateless Nature

- HTTP is a stateless protocol, meaning each request is independent
- The server does not retain information about previous client requests
- Session management bridges this gap, allowing servers to maintain user state across multiple requests

Session Objects

- Created per user/client connection
- Stores user-specific data (login info, preferences, shopping cart)
- Accessed via request.getSession()
- Methods: setAttribute(), getAttribute(), removeAttribute(), invalidate()

Session Lifecycle

01	02
Creation First call to getSession() creates a new session	Active Session persists across multiple requests
03	04
Timeout Inactive sessions are automatically destroyed (default 30 minutes)	Invalidation Explicit session.invalidate() call

Cookies

- Small text files stored on client browser
- Sent with every request to the server
- Attributes: name, value, domain, path, expiry, secure, httpOnly
- Types: Session cookies (deleted when browser closes), Persistent cookies (stored on disk)

Session vs Cookies

Sessions

- Server-side storage
- Secure
- Larger data capacity

Sessions use cookies to track session ID

Cookies

- Client-side storage
- Limited size (4KB)
- Visible to user

Security Best Practices

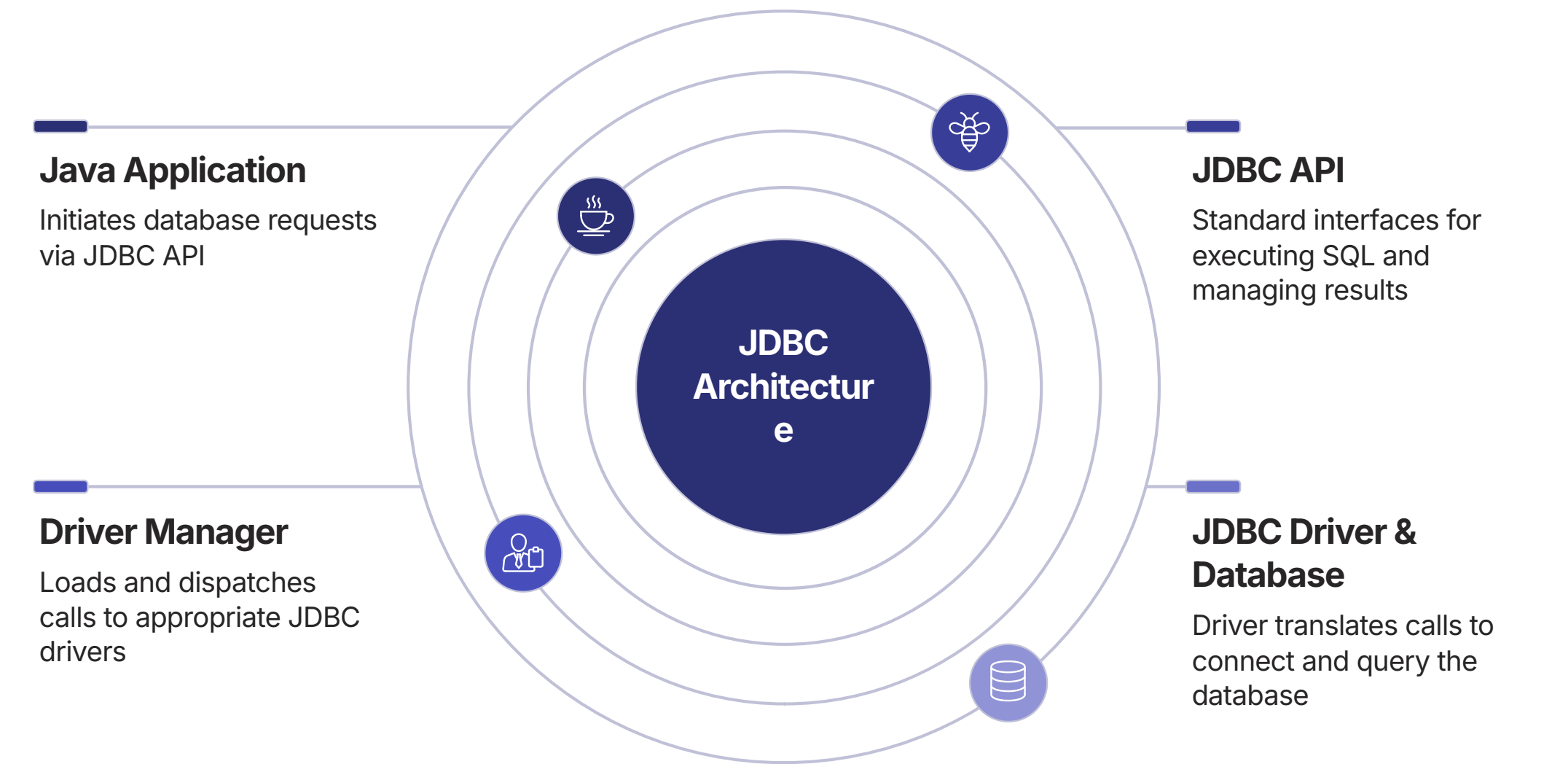
Use HTTPS for all session-related communication	Set HttpOnly flag on session cookies to prevent JavaScript access
Set Secure flag to transmit cookies only over HTTPS	Implement session timeout
Regenerate session ID after login	Validate session data on server side

JDBC & Database Connectivity

The Java Database Connectivity (JDBC) API is a standard Java API for connecting to relational databases from Java applications. It provides a common interface for interacting with various database systems, abstracting away the underlying implementation details. JDBC is crucial for enterprise applications requiring persistent data storage and retrieval.

JDBC Architecture & Components

JDBC consists of two main layers: the JDBC API and the JDBC Driver API. The API allows applications to connect to a data source, query the database, and process results. The Driver API is for vendors to implement JDBC drivers for their specific databases.



JDBC Drivers

Type 1: JDBC-ODBC Bridge

Uses ODBC driver to connect to the database. Not platform-independent and has performance overhead. Legacy.

Type 2: Native API (Partially Java)

Converts JDBC calls into native database API calls. Requires native client library on the client machine.

Type 3: Network Protocol (Pure Java)

Uses a middleware server to convert JDBC calls into vendor-specific database calls. Flexible but adds a layer.

Type 4: Native Protocol (Pure Java)

Directly converts JDBC calls into the database's native network protocol. Most common, high performance, and platform-independent.

Connection Pooling

Establishing a database connection is an expensive operation. Connection pooling manages a pool of pre-established, reusable database connections. When an application needs a connection, it requests one from the pool. After use, the connection is returned to the pool instead of being closed, significantly improving performance and scalability.

Basic JDBC Operations & Prepared Statements

Establishing Connection & Statement

The core of JDBC involves obtaining a `Connection`, creating a `Statement` or `PreparedStatement`, and executing SQL.

```
import java.sql.*;

public class JdbcExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String user = "root";
        String password = "password";

        try (Connection conn =
DriverManager.getConnection(url, user, password);
        Statement stmt = conn.createStatement()) {

            // Execute a simple query
            ResultSet rs = stmt.executeQuery("SELECT id, name
FROM users");
            while (rs.next()) {
                System.out.println("ID: " + rs.getInt("id") + ",
Name: " + rs.getString("name"));
            }
            rs.close(); // Close ResultSet
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Prepared Statements & Parameterized Queries

`PreparedStatement` is vital for security and performance. It pre-compiles SQL and allows parameterized input, preventing SQL injection attacks and improving execution speed for repeated queries.

```
// Using PreparedStatement
String sql = "INSERT INTO users (name, email) VALUES (?, ?)";
try (PreparedStatement pstmt =
conn.prepareStatement(sql)) {
    pstmt.setString(1, "Alice");
    pstmt.setString(2, "alice@example.com");
    int rowsAffected = pstmt.executeUpdate();
    System.out.println(rowsAffected + " row(s) inserted.");

    String updateSql = "UPDATE users SET email = ? WHERE
id = ?";
    PreparedStatement updatePstmt =
conn.prepareStatement(updateSql);
    updatePstmt.setString(1, "new.alice@example.com");
    updatePstmt.setInt(2, 1);
    updatePstmt.executeUpdate();
    updatePstmt.close();
} catch (SQLException e) {
    e.printStackTrace();
}
```

Best Practices: Exception Handling & Resource Management

1

Always Use Try-with-Resources

Automatically closes JDBC resources (`Connection`, `Statement`, `ResultSet`), preventing leaks. Introduced in Java 7.

2

Handle `SQLException`

JDBC operations can throw `SQLException`. Catch it and log details, as it provides specific error codes and messages.

3

Use Prepared Statements

Prevent SQL injection vulnerabilities and improve performance for frequently executed queries.

4

Implement Connection Pooling

Essential for high-performance applications to manage and reuse database connections efficiently.

5

Validate User Input

Even with prepared statements, always validate user input on the server side before processing or storing.



JSP Fundamentals & Syntax

JavaServer Pages (JSP) technology allows developers to create dynamic, platform-independent web pages by embedding Java code directly within HTML. It simplifies the development of web-based applications, offering advantages over traditional Servlets for presentation logic.

Why JSP? Advantages over Servlets

Separation of Concerns

Facilitates clear separation of presentation (HTML) from business logic (Java), leading to more maintainable code.

Faster Development

Changes to the presentation layer can be made without recompiling Java code, speeding up the development cycle.

Automatic Compilation

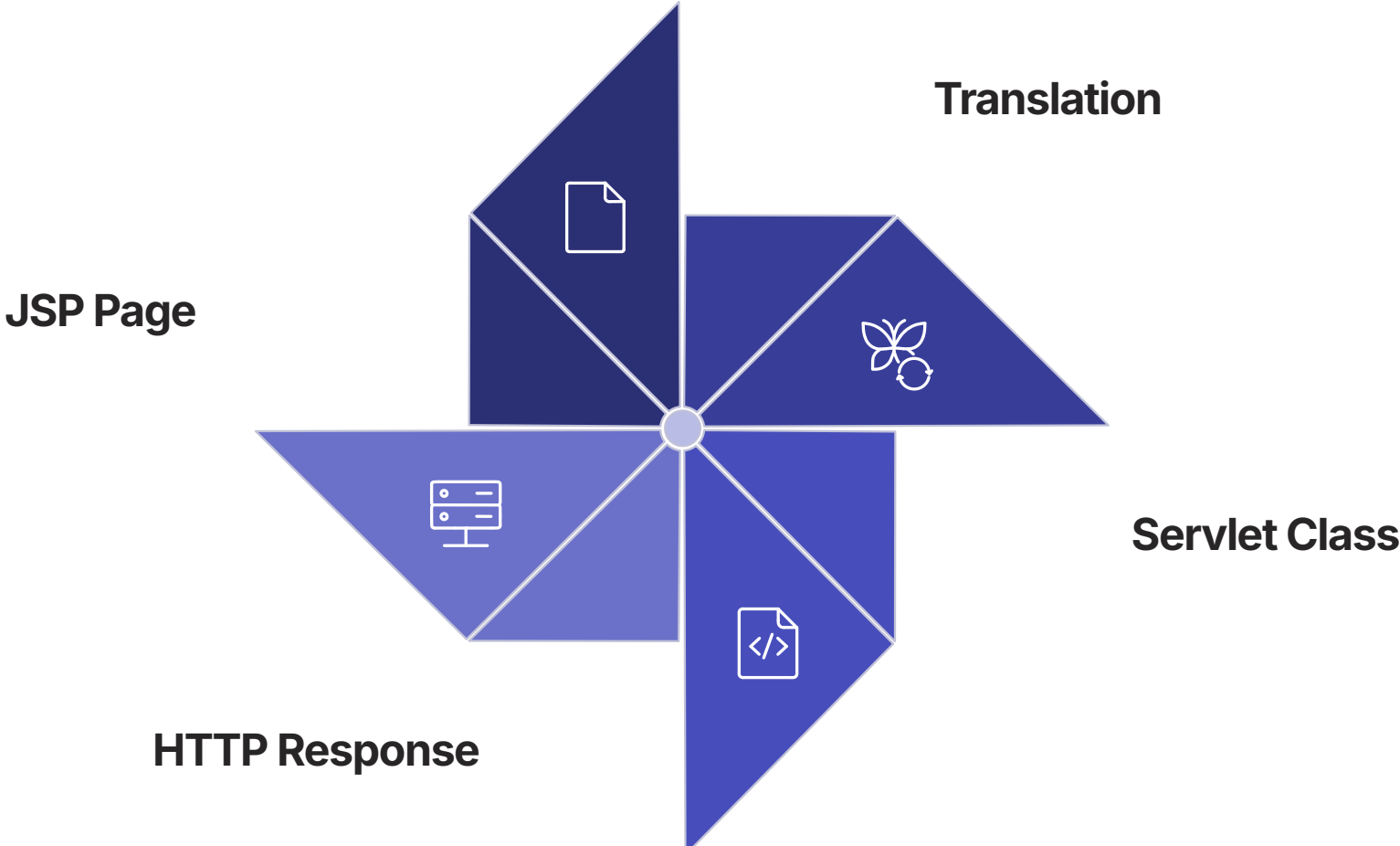
The web container automatically translates JSP files into servlets and compiles them, abstracting away manual steps.

Implicit Objects

Provides several pre-defined objects (e.g., request, response, session) readily available for use, reducing boilerplate code.

JSP Page Structure & Lifecycle

A JSP page combines static HTML content with dynamic Java components. The web container translates a JSP file into a Java Servlet, compiles it, and then executes it to generate the final response. This lifecycle involves translation, compilation, and execution phases.



During translation, the JSP container converts the .jsp file into a .java source file (a servlet). This servlet is then compiled into a .class file. Finally, this compiled servlet is executed by the server, and its output is sent back to the client as an HTTP response.

JSP Directives

<%@ page ... %>

Defines page-dependent attributes like content type, import statements, session usage, and error pages.

```
<%@ page language="java"
contentType="text/html;
charset=UTF-8"
pageEncoding="UTF-8"
import="java.util.Date" %>
```

<%@ include ... %>

Includes content from another resource (HTML, JSP, text file) at translation time, effectively merging files.

```
<%@ include file="header.html"
%>
```

<%@ taglib ... %>

Declares a tag library, specifying a prefix and URI to use custom tags within the JSP.

```
<%@ taglib
uri="http://java.sun.com/jsp/jstl/co
re" prefix="c" %>
```

JSP Scripting Elements

Declarations <%! ... %>

Used to declare variables and methods that become members of the generated servlet class. This code runs only once when the servlet is initialized.

```
<%! public int counter = 0; %>
<%! public String getGreeting() {
    return "Hello, JSP!";
} %>
```

Scriptlets <% ... %>

Allows embedding Java code directly into the JSP. This code is executed every time the JSP page is requested. Avoid complex logic here.

```
<%
String user = request.getParameter("name");
if (user != null && !user.isEmpty()) {
    out.println("Welcome, " + user + "!");
} else {
    out.println("Please enter your name.");
}
%>
```

Expressions <%= ... %>

Used to evaluate a Java expression and convert the result to a String, which is then inserted directly into the output HTML. No semicolon needed.

```
Current time: <%= new java.util.Date() %>
The counter is: <%= counter++ %>
```

JSP Comments

JSP supports two types of comments: JSP comments, which are ignored by the server and not sent to the client, and HTML comments, which are visible in the client's browser source.

```
<%-- This is a JSP comment. It will not be sent to the client.
--%>
<!-- This is an HTML comment. It IS visible in the client's
browser source. -->
```

JSP Implicit Objects

JSP provides several objects that are automatically created and available for use within scriptlets and expressions without explicit declaration.

**request**

HTTP request information (parameters, headers, cookies).

**response**

HTTP response management (setting headers, status codes).

**session**

User-specific data across multiple requests (HttpSession object).

**application**

Global data shared by all users across the entire web application (ServletContext).

**out**

JSPWriter object for sending content to the client's browser.

**pageContext**

Provides access to all namespaces and attributes of the page, request, session, and application scopes.

JSP Action Tags

JSP action tags are XML-based tags used to control the flow between pages, manage beans, and include resources at runtime.

1

<jsp:include page="..." />

Includes the output of another resource (JSP, HTML, Servlet) into the current page at request time. Each included page is processed independently.

```
<jsp:include page="footer.jsp" />
```

2

<jsp:forward page="..." />

Transfers control from the current JSP page to another resource (JSP or Servlet) on the server. The original request is passed to the forwarded page.

```
<jsp:forward page="login.jsp" />
```

3

<jsp:useBean id="..." class="..." />

Locates or instantiates a JavaBean and makes it available within a specified scope. Simplifies access to Java objects from JSP.

```
<jsp:useBean id="user" class="com.example.UserBean" scope="session" />
<jsp:getProperty name="user" property="username" />
```

Best Practices for JSP Development

- Avoid Scriptlets:** Minimize Java code within JSPs. Prefer Expression Language (EL), JSTL (JSP Standard Tag Library), and JavaBeans for cleaner, more readable code.
- Use MVC Architecture:** Separate presentation (JSP), business logic (JavaBeans/services), and control (Servlets) for robust and scalable applications.
- Validate User Input:** Always validate data on the server-side to prevent security vulnerabilities and ensure data integrity.
- Handle Errors Gracefully:** Implement custom error pages using `<%@ page errorPage="..." %>` to provide a better user experience.
- Secure Your Pages:** Be aware of common vulnerabilities like Cross-Site Scripting (XSS) and SQL Injection (even with EL/JSTL, proper encoding and parameterization are key).

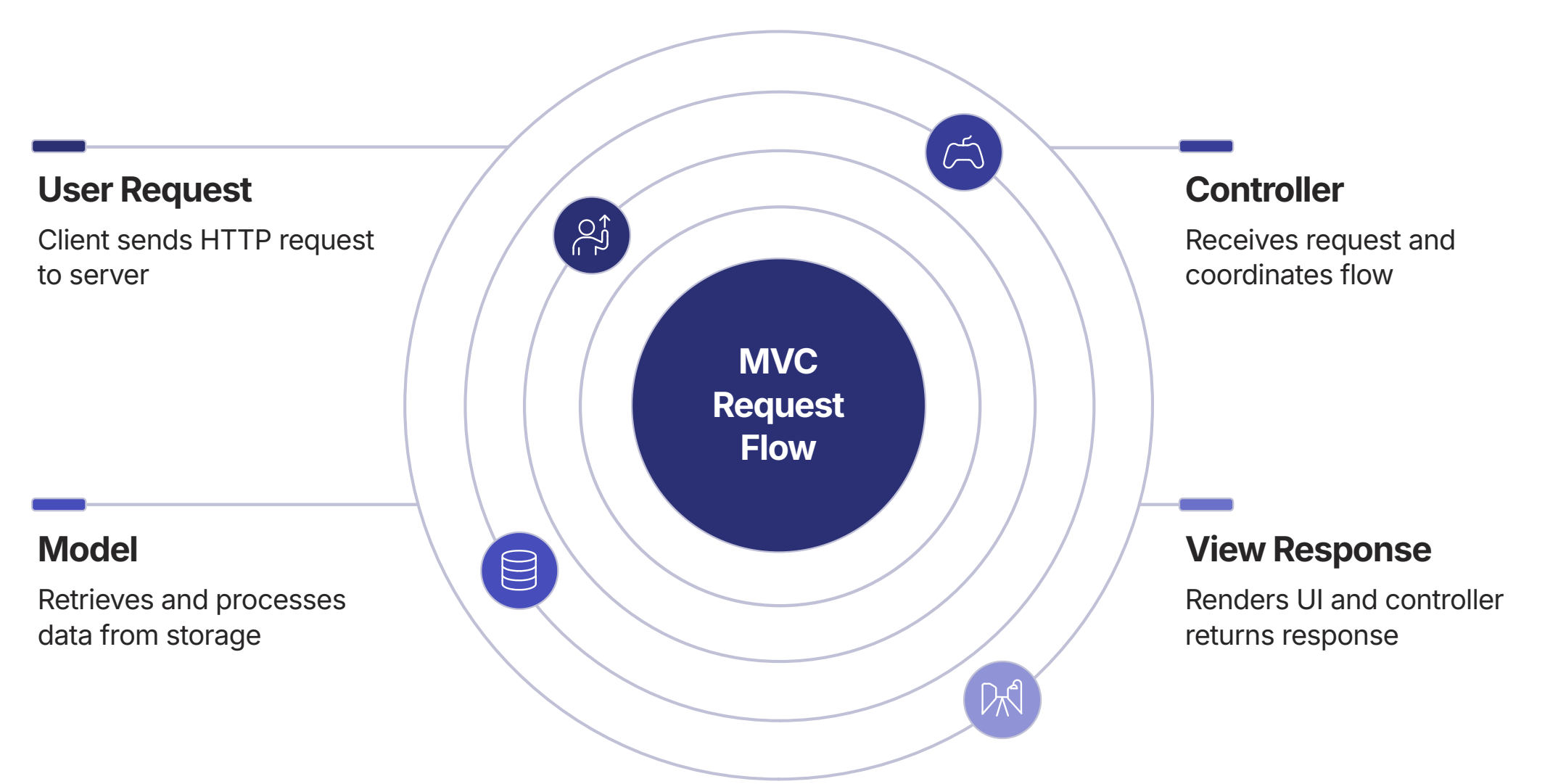
MVC Architecture & Best Practices

The Model-View-Controller (MVC) is an architectural pattern that separates an application into three main logical components: Model, View, and Controller. This separation aims to isolate business logic from user interface considerations, significantly improving modularity, scalability, and maintainability in web applications.

Core Components of MVC

1	Model (Business Logic & Data) The Model represents the application's core business logic, data, and rules. It manages the data, responds to requests from the Controller, and notifies the View(s) if its state changes. It is entirely independent of the user interface. In a Java web application, this often involves JavaBeans or EJBs.
2	View (Presentation Layer) The View is responsible for rendering the data received from the Model to the user in a readable format, typically HTML. It presents information and captures user interactions. Technologies like JSP are commonly used to implement the View, displaying dynamic content without containing business logic itself.
3	Controller (Request Handling) The Controller acts as an intermediary, receiving user input (e.g., HTTP requests), processing it, and orchestrating interactions between the Model and View. It interprets user actions, delegates tasks to the Model, and then selects the appropriate View to display the result. Servlets are typically used as Controllers in Java web applications.

Real-World MVC Flow



This diagram illustrates a typical user request lifecycle in an MVC application. The user initiates an action, which the Controller intercepts. The Controller then interacts with the Model to perform necessary operations or fetch data. Finally, the Controller chooses a View to present the results, and the View renders the final response back to the user.

Benefits of MVC Architecture

- Separation of Concerns**
Clearly separates business logic, data, and presentation, making the application easier to understand, develop, and maintain.
 - Enhanced Testability**
Individual components (Model, Controller) can be tested independently, simplifying unit testing and debugging.
- Improved Maintainability & Extensibility**
Changes to one component (e.g., UI updates in the View) have minimal impact on others, allowing for easier updates and feature additions.
 - Support for Parallel Development**
Different teams can work on the Model, View, and Controller simultaneously, speeding up development cycles.

Beyond MVC, enterprise applications often leverage additional design patterns like **Data Access Objects (DAOs)** for robust data layer abstraction, **Service Layer** for business logic encapsulation, and **Dependency Injection** for managing component relationships, further enhancing modularity and reusability.

Web Application Security

Safeguarding web applications against cyber threats is paramount for protecting sensitive data, maintaining user trust, and ensuring system integrity. Understanding common vulnerabilities and implementing robust security practices are critical for every developer.

Common Vulnerabilities & Mitigation Strategies

SQL Injection

Attackers manipulate database queries by injecting malicious SQL code through user input, leading to unauthorized data access, modification, or deletion.

- Prevention:** Use PreparedStatement with parameterized queries. Avoid dynamic SQL concatenation. Implement input validation.

```
String query = "SELECT * FROM users WHERE username = ? AND password = ?";
PreparedStatement ps = conn.prepareStatement(query);
ps.setString(1, username);
ps.setString(2, password);
```

Cross-Site Scripting (XSS)

Malicious scripts are injected into web pages viewed by other users, allowing attackers to steal session cookies, deface websites, or redirect users to malicious sites.

- Prevention:** Always perform output encoding (HTML entity encoding) for all user-supplied data displayed on a page. Use security headers like Content Security Policy (CSP).

```
<%
out.println(HTMLEncode(userInput)); %>
```

Cross-Site Request Forgery (CSRF)

Attackers trick authenticated users into submitting unwanted requests to a web application. The application processes the request believing it originated from the legitimate user.

- Prevention:** Implement anti-CSRF tokens (unique, random tokens associated with each user session and request). Use SameSite cookie attribute.

Essential Security Measures

Authentication & Authorization

Implement strong authentication mechanisms (e.g., multi-factor authentication) and granular authorization controls based on user roles and permissions.

Secure Password Handling

Never store passwords in plain text. Use strong, one-way hashing algorithms (e.g., Argon2, bcrypt) with salting. Enforce complex password policies.

HTTPS and SSL/TLS

Encrypt all communication between clients and servers using HTTPS to protect data in transit from eavesdropping and tampering.

Input Validation & Output Encoding

Validate all user input on both client and server sides to prevent malicious data from entering the system. Encode all output before displaying it to users.

Security Best Practices for Servlets and JSP

Minimize code in JSPs, utilize security frameworks, configure web.xml for secure session management and error handling, and regularly update dependencies.

Practical Examples & Code Snippets

To solidify understanding of Java web development concepts, let's explore practical code examples demonstrating key functionalities from Servlets, JSPs, and JDBC, along with essential best practices.

Servlet: Handling GET & POST

A basic servlet demonstrating how to process both GET and POST requests. GET requests typically retrieve data, while POST requests send data to the server for processing.

```
// MyServlet.java
@WebServlet("/mydata")
public class MyServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response) throws ServletException,
        IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h1>GET Request Processed!</h1>");
        // ... retrieve data for display
    }

    protected void doPost(HttpServletRequest request,
        HttpServletResponse response) throws ServletException,
        IOException {
        String input = request.getParameter("userInput");
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h1>POST Request Received! Input: " +
            input + "</h1>");
        // ... process data, save to DB etc.
    }
}
```

JSP Form with Validation

A JSP page that presents a simple form and includes basic server-side validation. This example shows how to display validation messages back to the user without redirecting.

```
<!-- form.jsp -->
<h1>User Registration</h1>
<% String error = (String) request.getAttribute("error");
%>
<% if (error != null && !error.isEmpty()) { %>
    <p style="color: red;"><%= error %></p>
<% } %>
<form action="register" method="post">
    Name: <input type="text" name="userName" /><br/>
    Email: <input type="text" name="userEmail" /><br/>
    <input type="submit" value="Register" />
</form>
```

JDBC: Simple Database Query

Connecting to a database using JDBC to execute a simple SELECT query. This snippet illustrates basic connection, statement execution, and result set processing.

```
// DBUtil.java (partial)
public class DBUtil {
    private static final String DB_URL =
        "jdbc:mysql://localhost:3306/mydb";
    private static final String USER = "root";
    private static final String PASS = "password";

    public static List<String> getUserNames() {
        List<String> names = new ArrayList<>();
        try (Connection conn =
            DriverManager.getConnection(DB_URL, USER, PASS);
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT name
        FROM users")) {

            while (rs.next()) {
                names.add(rs.getString("name"));
            }
        } catch (SQLException e) {
            e.printStackTrace(); // Proper logging in production
        }
        return names;
    }
}
```

Session Management: User Login

Demonstrates how to use HTTP sessions to maintain user state after login. A session attributes are used to store and retrieve user information across multiple requests.

```
// LoginServlet.java (partial)
protected void doPost(HttpServletRequest request,
    HttpServletResponse response) throws ServletException,
    IOException {

    String username = request.getParameter("username");
    String password = request.getParameter("password");

    if (authenticate(username, password)) { // Dummy
authentication
        HttpSession session = request.getSession(); // Get
existing session or create new one
        session.setAttribute("loggedInUser", username);
        response.sendRedirect("welcome.jsp");
    } else {
        request.setAttribute("loginError", "Invalid
credentials");

    request.getRequestDispatcher("login.jsp").forward(reque
st, response);
    }
}
```

Robust Error Handling

Proper error handling is crucial. This snippet shows how to define error pages in web.xml to present user-friendly messages instead of raw stack traces.

```
<!-- web.xml -->
<error-page>
    <error-code>404</error-code>
    <location>/error404.jsp</location>
</error-page>
<error-page>
    <exception-type>java.lang.Throwable</exception-
type>
    <location>/errorGeneral.jsp</location>
</error-page>

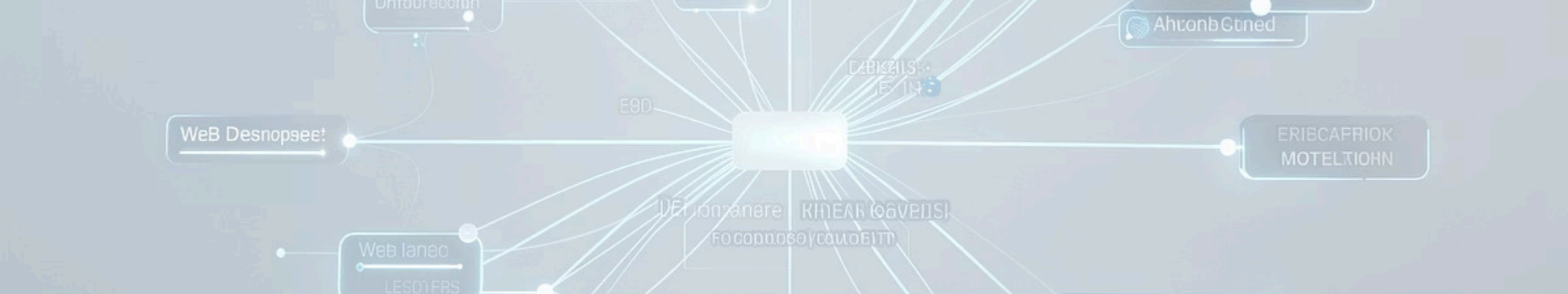
<!-- errorGeneral.jsp -->
<h1>Oops! Something went wrong.</h1>
<p>Please try again later or contact support.</p>
```

Best Practice: Utility Class

Encapsulate common, reusable logic in static utility methods. This promotes code reuse, reduces duplication, and makes code easier to test and maintain.

```
// AppUtils.java
public class AppUtils {
    public static boolean isValidEmail(String email) {
        // Basic email validation regex
        return email != null && email.matches("^[\w-
\.] + @ ([\w- ] + \. ) + [ \w- ] { 2, 4 } $");
    }

    public static String escapeHtml(String text) {
        // Basic HTML escaping to prevent XSS
        return text.replace("&", "&")
            .replace("<", "<")
            .replace(">", ">")
            .replace(""", """)
            .replace("'", "'");
    }
}
```



Summary & Review: Java Web Development

Reinforcing core knowledge for robust and scalable applications.

1	Key Concepts Recap Servlets: Foundation for HTTP request/response handling. JSP: Dynamic content generation, presentation layer. JDBC: Standard API for database connectivity.
2	Architecture Overview MVC Pattern: Separates application logic (Model), data presentation (View), and user input processing (Controller). Enhances maintainability and scalability.
3	Security Fundamentals Input Validation: Prevent SQL injection, XSS. Authentication/Authorization: Secure user access. Session Management: Protect session IDs.
4	Best Practices Checklist - Use parameterized queries (JDBC). - Leverage JSTL/EL in JSPs for cleaner code. - Implement robust error handling and logging. - Keep business logic out of Servlets/JSPs.
5	Common Pitfalls - Ignoring input validation. - Hardcoding sensitive information. - Mixing HTML/Java logic excessively in JSPs. - Lack of proper resource management (e.g., closing JDBC connections).
6	Further Learning - Official Oracle Java EE/Jakarta EE documentation. - Online courses (Coursera, Udemy, Pluralsight). - Books on Java web development and design patterns.
7	Exam Preparation Tips - Review API documentation thoroughly. - Practice coding examples for each concept. - Understand differences between container-managed vs. application-managed components. - Focus on security implications of various design choices.